

实验四：基于循环神经网络 GRU 的语音增强

一、实验简介

本实验的目的是搭建循环神经网络 RNN(Recurrent Neural Network)来实现语音降噪，将搭建一个简易的基于门控循环单元 GRU(Gated Recurrent Unit)的 RNN 网络对被 babble、car、train、restaurant、street 五种噪声污染的语音信号进行降噪处理。

本实验中用到的深度学习框架为 pytorch。Pytorch 的复杂度适中、介于 tensorflow 和 keras 之间，且能提供灵活的变化操作，适合作为 keras 入门后深入学习的选择。

在开始实验前需要安装好 python 以及相应的集成开发环境 IDE，主要是 pytorch、librosa(0.8 及以上版本)、torchaudio。

二、语音降噪

2.1 算法流程和原理

整体算法结构如图 1 所示：

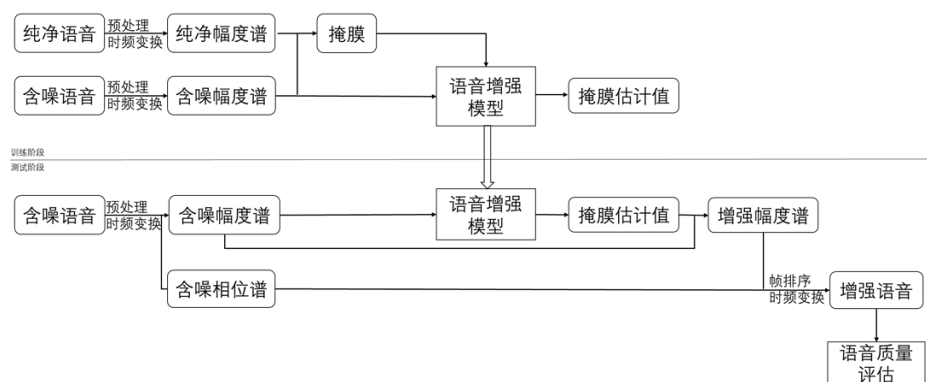


图 1 基于 RNN 的语音降噪算法结构

首先，需要对时频的语音信号进行分帧、加窗等预处理，再通过短时傅里叶变换 STFT(Short Time Fourier Transformation)将时域波形转换为时频特征。变换后的幅度谱作为 GRU 模型的输入特征，模型训练目标为理想比值掩膜 IRM(Ideal Ratio

Maksing), IRM 是纯净语音幅度谱与含噪语音幅度谱的比值, 在测试阶段, 估计的 IRM 和含噪语音幅度谱进行相乘, 得到纯净语音幅度谱的估计值, 同时利用含噪语音的相位用于重构纯净语音频谱, 再通过傅里叶逆变换, 得到纯净语音的时域波形。

STFT 的幅度谱作为本实验神经网络的输入特征, 用于估计每个频点的掩膜值, 该掩膜值是纯净语音与含噪语音的幅度谱比值。 $X(f,k)$ 和 $Y(f,k)$ 分别表示纯净语音信号和含噪语音信号的 STFT 幅度谱, 用 IRM 表征信号单元中纯净语音占整体的比例, 其计算表达式为:

$$IRM = \frac{|X(f,k)|^2}{|X(f,k)|^2 + |N(f,k)|^2} = \frac{|X(f,k)|^2}{|Y(f,k)|^2}$$

整个降噪过程中, 训练阶段需要为模型学习提供训练目标, 所以需要含噪语音与对应纯净语音的特征; 而测试阶段只需要输入含噪语音的幅度谱, RNN 利用含噪语音幅度谱估计 IRM, 利用 IRM 估计纯净语音的幅度谱, 通过含噪语音的相位重构纯净语音的频谱, 傅里叶逆变换后得到纯净语音的时域波形。

2.2 Noisy speech corpus (NOIZEUS)数据集

Noizeus 是 MIT 的 Loizou 实验室录制的语音数据库, 旨在用来对比不同的语音增强算法性能。NOIZEUS 包括一组纯净语音 (由三男三女讲述的共 30 个英文句子) 和其对应的被各种真实世界中噪声干扰后的、信噪比为 0、5、10dB 的含噪语音。

数据集已在工作目录 `./data/noizeus` 下, 本次实验使用数据集中的 `./clean` 和 `./withnoise` 两个目录下的语音文件, 通过以下方式进行数据集的加载。

```
filelist=[] # to store noisy audio
labellist=[] # to store clean audio

noisypath="./data/noizeus/withnoise/"
cleanpath='./data/noizeus/clean/'

noisyfolders=os.listdir(noisypath)
cleanfolders=os.listdir(cleanpath)

for folder in noisyfolders:
    audios=os.listdir(noisypath+folder+"/")
    for audio in audios:
        filelist.append(noisypath+folder+"/"+audio)
```

```
for audio in cleanfolders:
    labellist.append(cleanpath+audio)

print('read {} noisy files.'.format(len(filelist)))
print('read {} clean files.'.format(len(labellist)))
```

2.3 数据预处理

对于语音信号，往往不直接将时域波形当作特征 Feature 送入神经网络、而是应提取对应的时频表示。最常用的时频表示称为 STFT，其中 STFT 为一个复数矩阵，维度 FFT 点数 $\times$ 帧数。STFT 的提取需要人为指定一帧的样本数 frame(帧长 win)、相邻帧的重叠样本点数(hop)、FFT 的点数(默认等于帧长)。由于人耳对声音的感知主要由短时幅度谱决定，故将 STFT 的幅度谱这一重要特征称为语谱图(spectrogram)。

本实验取每句话的前 16704 个样本点，转换为一张(129,258)的语谱图，再将其分割为两张(129,129)的子语谱图。

```
def spectrogram(wav):
    win_length=256
    n_fft=256
    hop_length=64
    wav = wav.squeeze()
    S = torch.stft(wav, n_fft=n_fft, hop_length=hop_length,
                  win_length=win_length,
                  window=torch.hann_window(window_length=win_length),
                  center=True, return_complex=True)
    magnitude = torch.abs(S)      # 振幅
    phase = torch.exp(1j * torch.angle(S)) # 相位
    return magnitude, phase
```

2.4 准备 pytorch 框架中的数据集 Dataset

在 pytorch 中，对于原始数据信息，需要自定义一个 Dataset 类，来定义 pytorch 在训练时如何读取数据。自定义的 Dataset 类需要继承 Dataset 基类，并复写其中的 `__init__()`、`__getitem__()`、`__len__()` 三个特殊方法。

其中，`__init__()` 方法是创建该类的实例时会自动调用的一个初始方法；`__getitem__()` 方法接收一个参数 index，在方法中应定义 dataset 如何寻找第 index 个数据；`__len__()` 方法中应定义如何获取该 dataset 的大小。

在本实验中，第 `index` 个数据应当为一个由输入特征（一张(129,129)的语谱图）和目标特征（一个(129,129)的矩阵）组成的数据对。

#构建 pytorch 数据集

```
class MyDataset(Dataset):
    def __init__(self, filelist, labellist):
        super(MyDataset, self).__init__()
        self.filelist = filelist
        self.labellist = labellist
        self.win_length = 256
        self.hop_length = 64
        self.n_fft = 256

    def __len__(self):
        return len(self.filelist * 2)

    def __getitem__(self, idx):
        f = idx // 2
        n = idx % 2

        noised_waveform, _ = torchaudio.load(self.filelist[f])
        noised_waveform = noised_waveform[0][:16704] # 16704
        noised_waveform = noised_waveform.unsqueeze(0) # [1, 16704]

        clean_waveform, _ = torchaudio.load(self.labellist[int(filelist[f][-6:-4])-1])
        clean_waveform = clean_waveform[0][:16000] # 16704
        clean_waveform = clean_waveform.unsqueeze(0) # [1, 16704]

        noise_waveform = noised_waveform - clean_waveform

        noised_mag, _ = spectrogram(noised_waveform)
        clean_mag, _ = spectrogram(clean_waveform)
        noise_mag, _ = spectrogram(noise_waveform)

        postion = n
        length = 129
        sliced_noised = noised_mag[:, length * postion: length * (postion + 1)]
        sliced_clean = clean_mag[:, length * (postion): length * (postion + 1)]
        sliced_noise = noise_mag[:, length * (postion): length * (postion + 1)]
        # single_frame_noised_mag = sliced_noised[:, length: 2 * length]

        _eps = torch.finfo(torch.float).eps # 防止分母出现 0
        mask_IRM = torch.sqrt(sliced_clean ** 2 / (sliced_noise ** 2 + sliced_clean ** 2 + _eps))

# IRM, 模型输出
        label = mask_IRM # 掩蔽
        return sliced_noised.unsqueeze(0), label.unsqueeze(0)
```

覆写这三个方法后，我们就可以直接方便地按照下列语法取出想要的第 `index` 个数据对了，这里的[0]、[1]分别是数据对中的含噪谱图和真实谱图。

```
#查看 dataset 值
print(dataset[2][0].shape)
print(dataset[2][1].shape)
```

2.5 准备 pytorch 框架中的数据分发 DataLoader

理论上，只要给出要读取的数据的索引 `index`，就能从 `MyDataset` 中读出第 `index` 个数据对。

但是，实际在网络训练中，往往不是一个数据对、一个数据对地送入网络，而是将数据一批一批地送入网络。每批的个数就是 `batch size`。并且，往往我们还需要对数据打乱处理以避免过拟合。

在 `pytorch` 框架中，内置了负责一批一批地分发数据的 `Dataloader` 类。

首先，我们先分割训练数据集中的一部分作为验证集。

```
#按照 0.8 0.2 比例分割训练集和验证集

length=len(dataset)

train_size = int(0.8 * len(dataset))

validate_size = len(dataset) - train_size

train_set,validate_set=torch.utils.data.random_split(dataset,[train_size,validate_size])
```

然后再定义 `Dataloader` 类。它接收上文定义好的数据集 `dataset`，按照 `batchsize` 大小，一批一批地从 `dataset` 中取出数据、打包分发。

```
batch_size=16

train_dataloader = DataLoader(train_set, batch_size=batch_size,shuffle=True)

validate_dataloader = DataLoader(validate_set, batch_size=batch_size,shuffle=True)
```

三、循环神经网络 (RNN)

3.1 构建网络

在本实验中，使用一个八层的 RNN 来实现语音降噪任务。其输入为一张 129*129 的语谱图，即矩阵；其输出也为 129*129 的矩阵。注意到矩阵的两个维度是具有物理含义的，即时间轴和频率轴。因此，可以每次将语谱图中的一帧，即 129*1 的一个向量送入网络，经网络得到 129*1 的输出。再将同样的操作重复 128 次，共 129 个时间步。注意，RNN 与普通的全连接层的区别，正是体现在：网络每一层的参数也随着时间步不断更新。

```
#8 层 GRU 输入格式[batch channel sequence feature]

class RNN(nn.Module):

    def __init__(self):

        super(RNN, self).__init__()

        self.gru1=nn.GRU(129,hidden_size=1024,num_layers=1,batch_first=True)

        self.gru2=nn.GRU(1024,hidden_size=1024,num_layers=6,batch_first=True)

        self.gru3=nn.GRU(1024,hidden_size=129,num_layers=1,batch_first=True)

        self.sig=nn.Sigmoid()


    def forward(self, x):

        batch_size, n_channels, n_f_bins, n_frame_size = x.shape

        gru_in      =      x.reshape(batch_size,      n_channels      *      n_f_bins,
n_frame_size).permute(0, 2, 1)

        gru_out,_ = self.gru1(gru_in)

        gru_out,_ = self.gru2(gru_out)

        gru_out,_ = self.gru3(gru_out)

        gru_out  =  gru_out.permute(0,  2,  1).reshape(batch_size,  n_channels,
n_f_bins, n_frame_size)

        out=self.sig(gru_out)

        return out

model = RNN().to(device)
```

这样，就可以用 `model(input)`，直接获取网络的输出结果，框架会自动调用 `model` 的 `forward` 方法来前向传播。

在构建网络时，首先还应确定机器上可用的设备，代码如下。

```
device = "cuda" if torch.cuda.is_available() else "cpu"

print(device)
```

如果 GPU 可用，则 `device` 为 `'cuda'`，否则则为 `'cpu'`。在创建 `model` 实例后，应该调用 `to(device)` 方法，将它移动到规定的设备上（如果可用设备为 `cpu`，则实际上没有移动）。

3.2 定义 loss，优化器和 train 函数

先定义在训练要使用的 `loss` 函数和优化器，如下。

```
loss_fn = nn.MSELoss(reduction='mean')

optimizer = torch.optim.Adam(model.parameters(), lr=1e-4,) #损失函数值稳定后，将 lr 缩小十倍再次训练相同轮次
```

接下来定义 `train` 函数，在这个函数中完整执行训练过程。

```
def train(dataloader, model, loss_fn, optimizer):

    size = len(dataloader.dataset)

    model.train() #将模型设置为 train 模式

    for iteration, (X, y) in enumerate(dataloader):#dataloader 不断地分发出已经打包好的一批批 batch 数据

        X, y = X.to(device), y.to(device) #将输入数据移动到指定的设备上

        # 计算损失函数

        pred = model(X)

        loss = loss_fn(pred, y)#计算 loss 值

        # 反向传播的三个步骤

        optimizer.zero_grad()#由于 pytorch 的计算图不会自动清零，每次反向传播前要手动清
```

空

```
loss.backward()#根据计算图计算梯度值

optimizer.step()#梯度下降一次


if iteration % 10 == 0:

    loss, current = loss.item(), iteration * len(X)

    print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")
```

类似地，定义用于测试时使用的 `test` 函数。与 `train` 函数基本一致，区别在于 `test` 阶段不需要反向传播。

```
def test(dataloader, model, loss_fn):

    size = len(dataloader.dataset)

    num_batches = len(dataloader)

    model.eval()

    test_loss, correct = 0, 0

    with torch.no_grad():#由于不用反向传播，故没必要搭建计算图以节省内存、加快运行

        for X, y in dataloader:

            X, y = X.to(device), y.to(device)

            pred = model(X)

            test_loss += loss_fn(pred, y).item()

    test_loss /= num_batches

    print(f"Test Error: \n Avg loss: {test_loss:>8f} \n")
```

3.3 训练模型

模型训练若干 `epoch`，在普通笔记本的 GPU 上训练一轮大约需要四五分钟。参考程序设置为 5`epoch`，可以进行修改。

附件 `./model` 目录下已经事先保存了一个已训练 15 轮的模型 `rnn_gru.pt`，可以在此模型的基础上继续训练，即首先加载已有的模型参数，再进行 `epoch` 训练，也可以重新开始训练，两种训练方式的区别在于加载已有模型的指令是否注释。

```
# 如果需要加载已有模型参数，取消注释
# model.load_state_dict(torch.load('./model/rnn_gru.pt',map_location=device))

epochs = 5
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train(train_dataloader, model, loss_fn, optimizer)
    test(validate_dataloader, model, loss_fn)
print("Done!")
```

训练结束后，保存模型参数。这里使用的是只保存模型参数，不保存模型整体的方法，以节省空间。

```
if not os.path.exists('./model'):
    os.makedirs('./model')

torch.save(model.state_dict(), './model/rnn_gru.pt')
```

对应的加载模型的代码为：

```
model = RNN().to(device)

model.load_state_dict(torch.load('./model/rnn_gru.pt',map_location=device))
```

3.4 数据可视化

为了直观地验证降噪前后的效果，需要展示出训练前后的波形、语谱图，并可以播放试听。

绘制语谱图和播放试听的步骤封装在函数中，方便调用。

```
def plot_spectrogram(spec, title=None, ylabel='freq_bin', aspect='auto', xmax=None):
    fig, axs = plt.subplots(1, 1)
    axs.set_title(title or 'Spectrogram (db)')
    axs.set_ylabel(ylabel)
    axs.set_xlabel('frame')
    im = axs.imshow(librosa.power_to_db(spec), origin='lower', aspect=aspect)
    #im = axs.imshow(spec, origin='lower', aspect=aspect)
    if xmax:
        axs.set_xlim((0, xmax))
    fig.colorbar(im, ax=axs)
```

```
plt.show(block=False)
```

```
#播放试听
```

```
def play_audio(waveform, sample_rate):
```

```
    waveform = waveform.numpy()
```

```
    num_channels, num_frames = waveform.shape
```

```
    if num_channels == 1:
```

```
        display(Audio(waveform[0], rate=sample_rate))
```

```
    elif num_channels == 2:
```

```
        display(Audio((waveform[0], waveform[1]), rate=sample_rate))
```

```
    else:
```

```
        raise ValueError("Waveform with more than 2 channels are not supported.")
```

3.5 在新数据上生成预测结果

加载保存的模型并输入测试数据得到降噪后的语音信号。

```
#从测试音频中挑选一个进行降噪
```

```
num=3 #可以为 1, 2, 3
```

```
noised_waveform,_ = torchaudio.load('./data/mynoise{}.wav'.format(num))
```

```
noised_waveform=noised_waveform[:, :16704]
```

```
mag,phase=spectrogram(noised_waveform)
```

```
phase = phase.to(device)
```

```
mag1=mag[:, :129].to(device)
```

```
denoised_mag1=mag1*model(mag1.unsqueeze(0).unsqueeze(0))[0]
```

```
mag2=mag[:, 129:].to(device)
```

```
denoised_mag2=mag2*model(mag2.unsqueeze(0).unsqueeze(0))[0]
```

```
denoised_mag=torch.cat([denoised_mag1,denoised_mag2],dim=2)
```

```
complex_stft = (denoised_mag * phase).detach().cpu()
```

```
waveform=torch.istft(complex_stft, n_fft=256, hop_length=64, win_length=256,  
                    window=torch.hann_window(window_length=256),center=True)
```

降噪后的语音时域波形为 `waveform`，降噪前的含噪语音时域波形为 `noised_waveform`，绘制降噪语音、含噪语音的**时域波形**代码为：

```
plt.plot(waveform.squeeze().numpy())#降噪后波形
```

```
plt.plot(noised_waveform.squeeze().numpy())#降噪前波形
```

参考已有代码，绘制出降噪语音 `waveform`、含噪语音 `noised_waveform` 的语谱图，进行比较。需要学生在原始程序中加入**绘制语谱图**的代码：

根据降噪语音时域信号 `waveform`、含噪语音时域信号 `noised_waveform`，绘制语谱图

接下来定义播放函数，在 `jupyter` 交互式笔记本中播放，直观地感受降噪效果。
(注意：若使用 `pycharm` 或 `vscode` 等 IDE 打开 `ipynb` 格式文件，可能出现无法播放的情况。请使用 `jupyter` 打开即可)

播放降噪语音、含噪语音代码为：

```
def play_audio(waveform, sample_rate):
    waveform = waveform.numpy()

    num_channels, num_frames = waveform.shape
    if num_channels == 1:
        display(Audio(waveform[0], rate=sample_rate))
    elif num_channels == 2:
        display(Audio((waveform[0], waveform[1]),
rate=sample_rate))
    else:
        raise ValueError("Waveform with more than 2 channels are not
supported.")

play_audio(noised_waveform,8000)#8000 为语音信号的采样率
play_audio(waveform,8000)
```
